A

Project Synopsis

On

An Agentic AI Framework for Reliable and Evidence-Based Automated Code Review

Submitted by:

Name	PRN
Ms. Sakshi Bhosale	23610088

Under the guidance of

Mrs. B. S. Shetty

Department of Information Technology

Walchand College of Engineering,Sangli

Maharashtra, India. 416415

Title: An Agentic AI Framework for Reliable and Evidence-Based Automated Code Review

1. Introduction:

Software development involves continuous code changes that are commonly submitted through Pull Requests for review before integration. Code review plays a critical role in maintaining software quality, security, and reliability, but manual review requires significant developer effort and may overlook subtle defects.

Recent advances in Large Language Models (LLMs) have demonstrated their ability to understand and reason about source code. However, automated code review remains challenging because effective review often requires repository-level context, accurate defect identification, and reliable verification of generated findings. Recent research has specifically identified context selection, defect detection, false-alarm reduction, and integration with developer workflows as major challenges in automated code review.

The proposed project develops an Agentic AI Code Reviewer that combines multiple specialized AI agents with static-analysis and dependency-scanning tools. Instead of relying on a single LLM response, the system divides the review into specialized tasks and uses a verification stage to validate potential issues.

The system will integrate with GitHub Pull Requests, analyze changed code along with relevant repository context, identify potential defects, and generate structured review comments containing the issue, severity, evidence, confidence, and recommended remediation.

2. Problem Statement:

Modern software development relies on Pull Requests (PRs) and code reviews to identify defects before code is merged into a software project. However, manual code review is time-consuming and may fail to consistently detect security vulnerabilities, logical errors, inadequate error handling, missing tests, and dependency-related risks, particularly in large repositories.

Existing AI-based code review systems can analyze code and generate review suggestions, but a single Large Language Model (LLM) may produce false positives, miss important defects, hallucinate issues, or lack sufficient repository-level context. Static-analysis tools, on the other hand, can reliably detect predefined patterns but have limited understanding of program intent and complex logic.

3. Significance of Proposed work:

The proposed work aims to improve the efficiency, accuracy, and reliability of automated code review by using an Agentic AI-based approach for GitHub Pull Requests. The system will analyze code changes from multiple perspectives, including security vulnerabilities, logical defects, error handling, testing adequacy, and dependency risks. By combining Large Language Model (LLM) reasoning with static-analysis tools such as Semgrep and Bandit, the system can provide both contextual understanding and technical evidence. This reduces the dependency on manual review for identifying common issues and helps developers receive timely feedback before code is merged.

A key significance of the proposed system is its focus on reducing false positives and improving trust in AI-generated reviews. Instead of accepting every finding generated by an AI model, a dedicated verification agent will validate findings using the changed code, repository context, and analysis-tool results. The system will also incorporate failure handling for large Pull Requests and unsuccessful AI operations. Its performance can be evaluated using real historical software defects through metrics such as precision, recall, F1-score, false-positive rate, detection rate, review time, and cost. Thus, the project provides not only a practical AI-based code-review solution but also a measurable framework for studying the reliability of Agentic AI in software engineering.

4. Objectives:

1. To develop an automated GitHub Pull Request code-review system using Agentic AI.
2. To design a multi-agent architecture for analyzing security, logic/bug risks, error handling, and test adequacy.
3. To integrate LLM-based semantic reasoning with static-analysis tools such as Semgrep and Bandit.

4. To perform dependency scanning for detecting known vulnerabilities in third-party libraries.
5. To develop a verification mechanism that validates AI-generated findings and reduces false positives.
6. To implement failure-handling mechanisms such as retries and code chunking for large or complex Pull Requests.
7. To evaluate the proposed system against historical real-world software defects using quantitative performance metrics.

5. Methodology:

The **Agentic AI Code Reviewer** will be developed using an Agile methodology to ensure iterative development, continuous testing, and progressive improvement of the automated code-review process. The development process will be carried out in well-defined phases, enabling systematic integration of GitHub, Large Language Models, specialized AI agents, static-analysis tools, and verification mechanisms.

1. Requirement Analysis:

1. Analysis of automated code-review and Pull Request analysis requirements.
2. **Functional requirements:** GitHub Pull Request integration, code-difference extraction, security analysis, bug detection, error-handling analysis, test analysis, and review generation.
3. **Non-functional requirements:** accuracy, reliability, scalability, explainability, response time, and false-positive reduction.
4. Study of available LLMs, GitHub APIs, static-analysis tools, datasets, computational resources, and project constraints.

2. Design:

1. Modular agentic system architecture design.
2. Pipeline: Pull Request extraction, repository context collection, planning, specialized code analysis, evidence collection, finding verification, and final review generation.
3. Definition of data flow, agent responsibilities, model inputs/outputs, and review formats.
4. Integration strategy for LLM reasoning, static analysis, dependency scanning, and failure-handling mechanisms.

3. Backend Development:

1. Backend implementation using Python and FastAPI.
2. GitHub API and webhook integration for retrieving Pull Requests, changed files, and code differences.
3. Implementation of specialized AI agents for security, bug/logic, error-handling, and test analysis using an LLM.
4. Integration of LangGraph for agent orchestration and workflow management.
5. Integration of Semgrep, Bandit, and dependency-scanning tools for supporting code and security analysis.
6. Implementation of repository-context processing, finding verification, retry mechanisms, and code chunking for large Pull Requests.

4. Review Generation and Integration:

1. Development of a structured review-generation module.
2. Generation of findings containing issue type, severity, affected file/line, explanation, evidence, confidence, and recommended solution.
3. Verification of AI-generated findings using repository context and static-analysis results.
4. Integration with GitHub to automatically publish verified review comments on Pull Requests.

5. Testing:

1. Testing on Pull Requests containing different types of security, logic, error-handling, testing, and dependency-related issues.
2. Functional testing of individual agents and the complete review pipeline.
3. Testing of large Pull Requests, failed AI operations, and retry mechanisms.
4. Evaluation using historical software defects and comparison with known ground-truth fixes.
5. Performance evaluation using precision, recall, F1-score, false-positive rate, detection rate, review time, and API/model cost.
6. Analysis of system limitations, false positives, false negatives, and failure cases.

6. Block Diagram:

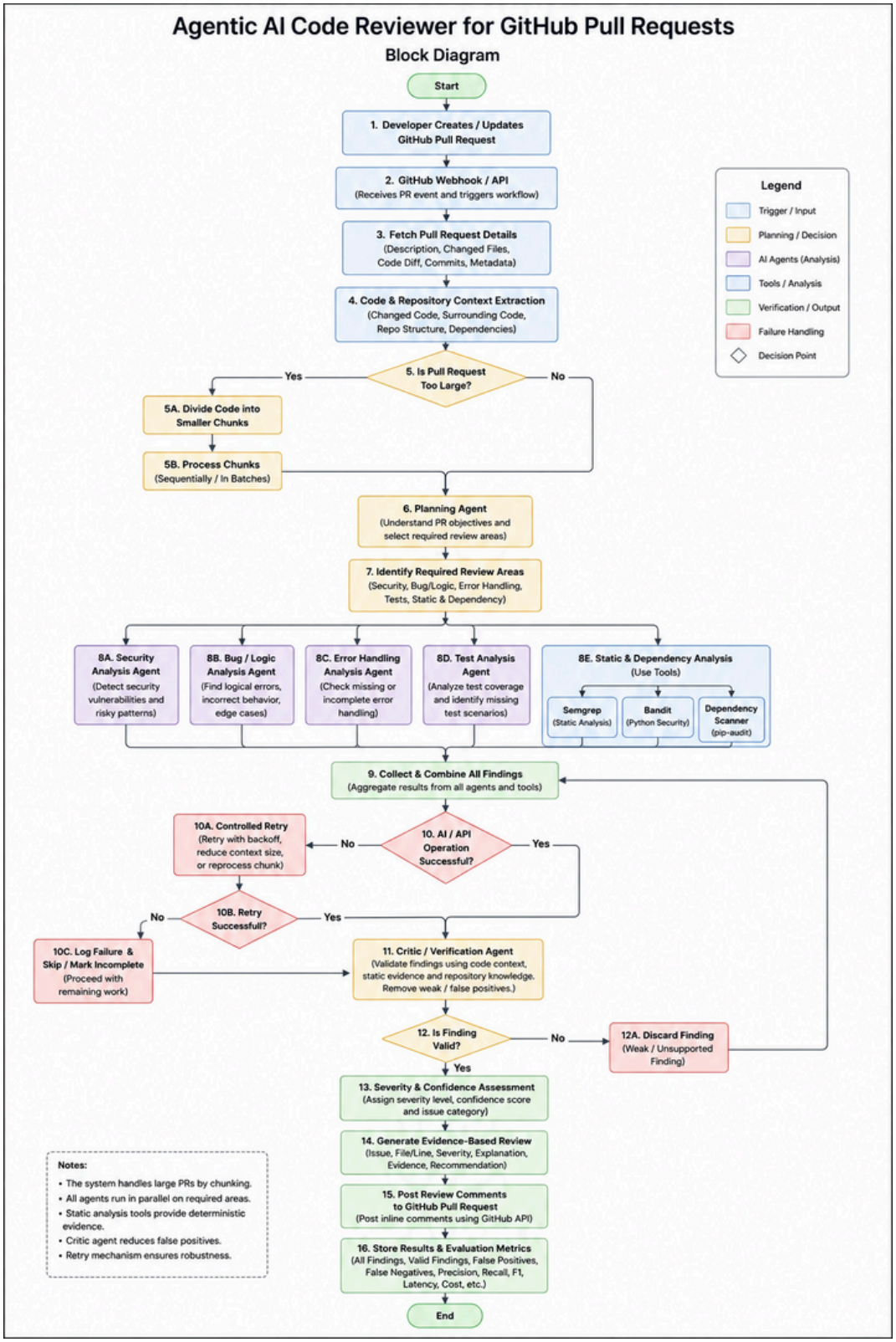


fig. Block Diagram of project

7. Project Plan:

Project plan is divided into the following phases mentioned below:

Phase 1: Requirement Analysis

1. Study existing AI-based and automated code-review systems.
2. Identify project requirements, objectives, scope, and feasibility.
3. Study relevant research papers, GitHub Pull Request workflows, and existing code-analysis approaches.
4. Identify suitable datasets and historical software defects for evaluation.

Phase 2: System Design

1. Design the overall Agentic AI code-review architecture and workflow.
2. Select LLMs, agent orchestration framework, static-analysis tools, and other technologies.
3. Design the modules for Pull Request analysis, repository context extraction, multi-agent review, verification, and review generation.
4. Define data flow, agent responsibilities, and review output format.

Phase 3: AI and Agent Development

1. Develop the GitHub Pull Request and code-context extraction module.
2. Implement specialized AI agents for security, bug/logic, error handling, and test analysis.
3. Implement LLM-based code reasoning and agent orchestration using LangGraph.
4. Integrate Semgrep, Bandit, and dependency-scanning tools for supporting analysis.

Phase 4: System Integration

1. Integrate all AI agents and analysis tools into a unified application.
2. Implement the critic/verification agent to validate findings and reduce false positives.
3. Implement failure handling, retry mechanisms, and code chunking for large Pull Requests.
4. Integrate the system with GitHub for automated review generation and Pull Request comments.

Phase 5: Testing, Evaluation, and Documentation

1. Test the system using real and historical Pull Requests containing known software defects.
2. Evaluate defect detection, false-positive reduction, and overall review performance.
3. Compare the proposed multi-agent approach with a single-LLM code reviewer using precision, recall, F1-score, and other evaluation metrics.
4. Optimize system performance and resolve identified issues.
5. Prepare the project report and technical documentation.

8. Technologies Used:

1.P**rogramming Language**

1. Python

2.AI / LLM Technologies

1. Large Language Model (LLM) API
2. LangGraph (Agent Orchestration)

3.Code Analysis and Security Tools

1. Semgrep
2. Bandit
3. Dependency Scanning – pip-audit / GitHub Dependency Scanning

4.AI-Based Code Review Modules

1. Security Analysis Agent
2. Bug / Logic Analysis Agent
3. Error Handling Analysis Agent
4. Test Analysis Agent
5. Critic / Verification Agent

5.Backend and API Technologies

1. FastAPI
2. GitHub REST API
3. GitHub Webhooks

6.Database and Supporting Technologies

1. PostgreSQL
2. Redis

7.Evaluation Metrics

1. Precision
2. Recall
3. F1-Score
4. False Positive Rate
5. False Negative Rate
6. Defect Detection Rate
7. Review Latency
8. API / Model Cost

8.Frontend / User Interface

1. React.js
2. TypeScript

9. References

A. Research Papers

[1] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, "RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing," Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR, vol. 267, pp. 21083–21100, 2025.
https://proceedings.mlr.press/v267/guo25n.html

[2] J. Lu, L. Jiang, X. Li, J. Fang, F. Zhang, L. Yang, and C. Zuo, "Towards Practical Defect-Focused Automated Code Review," Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR, vol. 267, pp. 40505–40536, 2025.
https://proceedings.mlr.press/v267/lu25f.html

B. Technical Documentation

[3] GitHub, "REST API Endpoints for Pull Requests," GitHub Documentation.
REST API endpoints for pull requests - GitHub Docs

[4] LangChain, "LangGraph Documentation – Agent Orchestration Framework," LangGraph Documentation.
https://docs.langchain.com/oss/python/langgraph/overview

Name	PRN
Ms. Sakshi Bhosale	23610088

Mrs. B. S. Shetty	**Dr. R. R. Rathod**	**Dr. U. B. Chavan**
Guide	**Class Teacher**	**Signature of HOD**